Kodama & network science in hypertext[1]

時 2025 年 10 月 23 日

[1]The title pays tribute to Jon Sterling's presentation for *Forester* a year ago.

Outline

Episode 1 ⊢ Genesis

Episode 1 ⊢ Genesis

§ Derived classic blog concepts

Let's first demonstrate that many concepts in traditional blogs, such as tags, categories, archives, and their respective indexing capabilities, are actually specialized functionalities.

Taking the so-called **tag** function as an example, the other ones are also completely similar:

- `Hugo` https://adityatelange.github.io/hugo-PaperMod
- `Hexo` https://probberechts.github.io/hexo-theme-cactus/cactus-white/public/
- `Zola` https://serene-demo.pages.dev

Episode 1 $\vdash$ Genesis

The tag function can be summarized into the following two things:

1. Each post has several tags, with a fixed post X_i. The tag A_j within X_i is a link that points to a page displaying all pages that have used tag A_j.
2. There is a tag index page called **tags**, which displays all tags $\{A_i\}_{i\in I}$.

Fact.

$$\begin{tikzcd} \text{Tag / Category / Archive} \quad = \quad X_i : \text{Post} \arrow[r, bend left, "1:n"] & A_i : \text{Tag} \arrow[l, bend left, "n:1"] \; + \; \text{Tag Index} \end{tikzcd}$$

$$= \; (X_i \rightleftarrows A_i, A_i \rightleftarrows \text{Tags})$$

Episode 1 $\vdash$ Genesis

```
---
tag1: [Rename1](/tags/tag1.md)
tag2: [Rename2](/tags/tag2.md)
---
```

X_i : `post1.md`

```
---
title: Tag Name
index: [](./index.md)
---
```

A_j : `/tags/tag1.md`

```
---
title: Tags
---
```

Tags: `/tags/index.md`

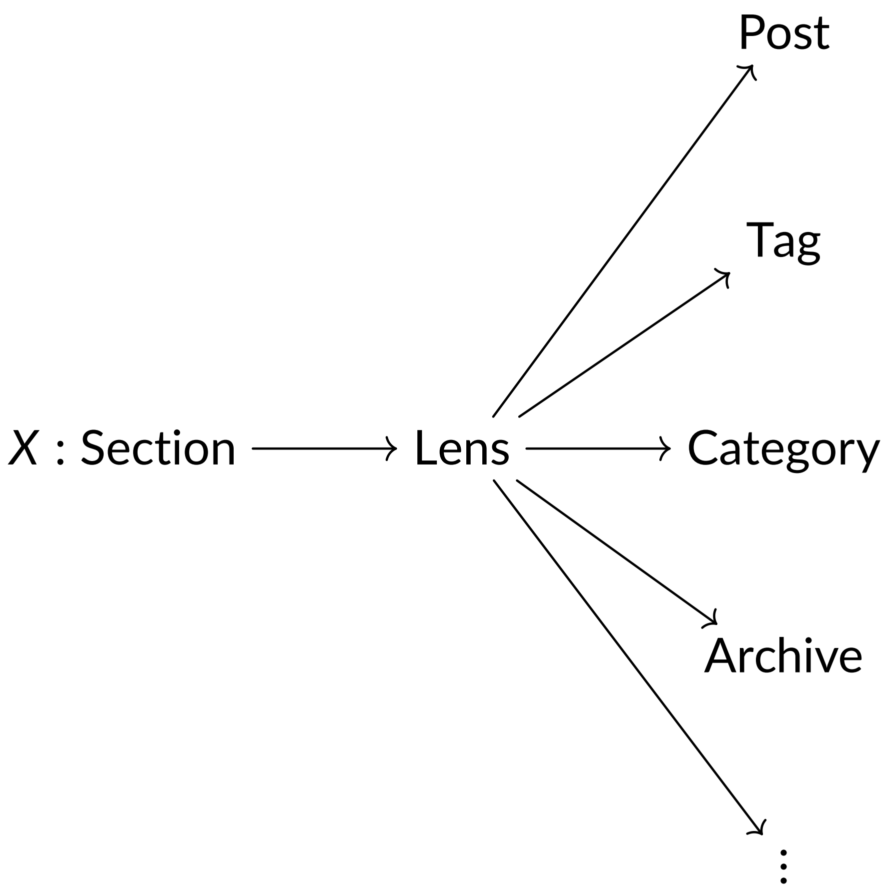
- A_j-Backlinks: all posts with tag A_j e.g., X_i.
- Tags-Backlinks: all tags with `[](./index.md)`.

Episode 1 $\vdash$ Genesis

This actually expands the functionality of tags, as the use of tags allows for

1. local renaming.
2. hiding or display in tag index page.
3. explaining the meaning of the tag.
4. etc.

In short, every tag page is now considered as the *__first class object__*, just like a post.



Episode 1 ⊢ Genesis

§ Why KaTeX + Typst

KaTeX has a very rudimentary support for commutative diagrams built-in, by emulating the `amscd` package. Unfortunately, this support is completely inadequate for usage by mathematicians today:

1. Only square diagram shapes are supported: *commutative diagrams in general have diagonal and curved lines, but these are not supported.*
2. *The rendering of the limited gamut of supported commutative diagrams is broken in most browsers (at least Safari and Firefox). In particular, lines are jagged as they are pieced together from pipes and arrows that are subtly misaligned.*

— Jon Sterling

https://www.forester-notes.org/tfmt-000J/index.xml

Episode 1 ⊢ Genesis

§ Scientific writing in hypertext

I will continue to quote Jon Sterling's insights. Without a doubt, there is currently no more *rigorous and clear* understanding of the concept of **"forest"** on the internet.

Atomicity of scientific notes https://www.forester-notes.org/tfmt-0007/index.xml

- a note should capture just one thing, and if possible, all of that thing.
- A related point is that it should be possible to understand a note by *reading it*, and *traversing the notes* that it links to and *recursively understanding* those notes.
- Traditional mathematical writing … understanding the meaning of a particular node (e.g. a theorem or definition) usually requires understanding everything that came (textually) before it.
- we should prefer explicit context over implicit context.

Episode 1 ⊢ Genesis

It is often difficult to meet the following requirements, and this principle is designed *very ideally*. In practical operation, writers often need to *repeatedly refactor* their notes to approach these effects.

Achieving atomicity https://www.forester-notes.org/tfmt-0008/index.xml

- no free variables: do not rely on one-off objects that are defined incidentally upwards in the hierarchy; turn them into atomic nodes that can be linked;
- favor explicit dependency: whenever using a terminology or construction that has been defined elsewhere, link it;
- notation should be decodable: all notations (except the most very basic) should be recalled via a link.

Episode 2 ⊢ Infra moonbit.community

§ Journal, blog, notes and wiki

Background

Unfortunately, for various reasons, these static site generators all have issues when it comes to mathematical writing, which severely limits expressive power.

Blogs, wikis, and personal notes may appear to be different types of web pages, but with the concept of *sections* and *backlinks*, they will be **unified**.

Presentations www.forester-notes.org/ ··· .pdf

Forester is your lecture notes or your public wiki or your private lab journal or your blog.

§ Schematical issue

Hierarchical structure Markdown, As a popular markup language with multiple incompatible standards, it is mostly regarded as a relatively more cumbersome HTML generation language However, all Markdown[2] inevitably suffers from the problem of hierarchical structure.

Grammar Fusion The expression of formulas is completely embedded in the markdown language, which combines TeX class syntax and HTML content syntax inherited from other markup languages by markdown.

Trusted Extension JavaScript has become the de facto standard for web development, but with the development of hardware and software technology, JavaScript has too many characteristics that are no longer suitable for this era.

---

[2] Even if codeblocks, headers, and some extension syntax e.g., `:::` are abused, it is still not feasible to freely express hierarchical structures.

Episode 2 ⊢ Infra moonbit.community

> For example, without the module concepts designed and promoted by CommonJS, AMD, and ESM, it is almost difficult to develop large-scale programs using JavaScript Its modern typed version TypeScript faces similar issues as C++and C in the past: it must be forced to be compatible with poor JavaScript features at the underlying level.

But the emergence of WASM and *MoonBit* provided a promising solution to this matter.

Give back to LLM

Arise, symbolic AI!

— *Jon Sterling*

Arise, MoonBit!

— 网络

Episode 3 $\vdash$ Analogues

Episode 3 ⊢ Analogues

§ From *n*Lab to 1Lab

*n*Lab and 1Lab are basically the final effects that can be presented by *linking* and *embedding* functions, but in terms of the tendency towards solidification, because 1Lab defines and derives all items in *formal language* i.e., code. This tendency of 1Lab is much stronger.

The Purpose of nLab https://ncatlab.org

- *n*Lab is a wiki for collaborative work on Mathematics, Physics, and Philosophy — especially (but far from exclusively) from the higher structures point of view: *with a sympathy towards the tools and perspectives* of homotopy theory / algebraic topology, homotopy type theory, higher category theory and higher categorical algebra.
- A fundamental idea behind the *n*Lab is that the *linking between pages of a wiki is a profound way to record knowledge*, placing it in a *rich context*.

Episode 3 ⊢ Analogues

The Idea of 1Lab https://1lab.dev

- 1Lab, A formalised, cross-linked reference resource for mathematics done in Homotopy Type Theory. Unlike the HoTT book, the 1lab is not a "linear" resource: *Concepts are presented as a directed graph*, with *links indicating dependencies*.
- 1Lab is an experiment in discoverable formalisation: an extensive library of formalised mathematics, presented as an explorable reference resource. Our implementation of a mathematical library, using literate source code, has allowed us a dual approach to the explanation of mathematical concepts: the code and the prose are complementary, and everyone is presented with the opportunity to choose their own balance between the rigid formalisation and the intuitive explanations.

Episode 3 $\vdash$ Analogues

§ $\mathrm{Hom}(X, -) \leftarrow X \rightarrow \mathrm{Hom}(-, X)$

We have actually come into contact with Forester or Kodama's *philosophy of solidification* in many places, such as

- Static analysis: For each item, display its *definition* and *references*.
- Dependency analysis: For each module, calculate its' *"imports"* and *"imported by"*.

This actually provides two completely different ways to *define* or *understand* an object.

Episode 3 $\vdash$ Analogues

At a very rough level, we can think of *viewing the definition of an object as understanding how to construct it*, and for types, this is the *constructor*. We can use examples from the category of sets that high school students can fully understand, implying this through *cardinal arithmetic*.

$$|\mathrm{Hom}(\{\mathrm{pt}\}, X)| \;=\; |X|, \qquad |\mathrm{Hom}(A, X)| \;=\; |X|^{|A|}$$

More examples that high school students can understand:

- `data Bool := True | False`.
- ```
  inductive Nat where
    | zero : Nat
    | succ : Nat → Nat
  ```

So what is the natural number 2? Our definition of the natural number 2 now is:

```
succ (succ zero)
```

Then we can easily see that the *constructor* is exactly what $\mathrm{Hom}(-, X)$ is trying to capture.
```

Episode 3 $\vdash$ Analogues

Let's shift our perspective to another place.

In fact, if a high school student also tries to understand why this method is useful from the perspective of *cardinal arithmetic*, they may encounter some difficulties. But programming languages once again provide a shortcut to explore the mechanisms.

If we have

$$\frac{\Gamma}{\texttt{apple : Fruit}}, \qquad \frac{\Phi}{\texttt{getColor : Fruit} \to \texttt{Color}}$$

then we will get

$$\frac{\Gamma, \Phi}{\texttt{getColor apple : Color}}$$

Episode 3 $\vdash$ Analogues

With enough of these *projection* methods, we can also obtain any information about the object. At this point, it can be considered that *viewing the definition of an object is to understand all the uses of that object*. i.e.,

$$\mathrm{Hom}(X, -)$$

More examples that high school students can understand:

- $\mathrm{Hom}(X, \mathrm{Bool}) = 2^X$.
- If for all finite sets Y, $|\mathrm{Hom}(X, Y)| = |Y|^n$, then $|X| = n$.
- In generally, Hom can be considered as the way in which all elements of X are *dyed* into the color of Y.

That's why even when designing things like static site generators, we should respect *duality*.

Thanks!

To see an example, go to: https://kokic.github.io/riemann-surfaces.